



Pedidos ERP

MANUAL TÉCNICO

Sistema de Pedidos y gestión empresarial

Arquitectura, instalación, configuración, seguridad, modelo de datos,
API, reglas de negocio, pruebas y mantenimiento.

Versión 2.0 · rama *feature/login-erp*

Fecha 2 de octubre de 2026

Dirigido a desarrolladores, administradores de sistemas y soporte técnico

Contenido

1 Introducción y alcance

2 Arquitectura

3 Tecnologías y versiones

4 Estructura del repositorio

5 Instalación y despliegue

Docker · variables de entorno · primer arranque · desarrollo local · operación diaria

6 Backend (API .NET 8)

7 Seguridad

8 Roles y permisos

9 Reglas de negocio

10 Base de datos

11 API REST: referencia de endpoints

12 Frontend (Flutter Web)

13 Pruebas automatizadas

14 Mantenimiento, extensión y solución de problemas

1 Introducción y alcance

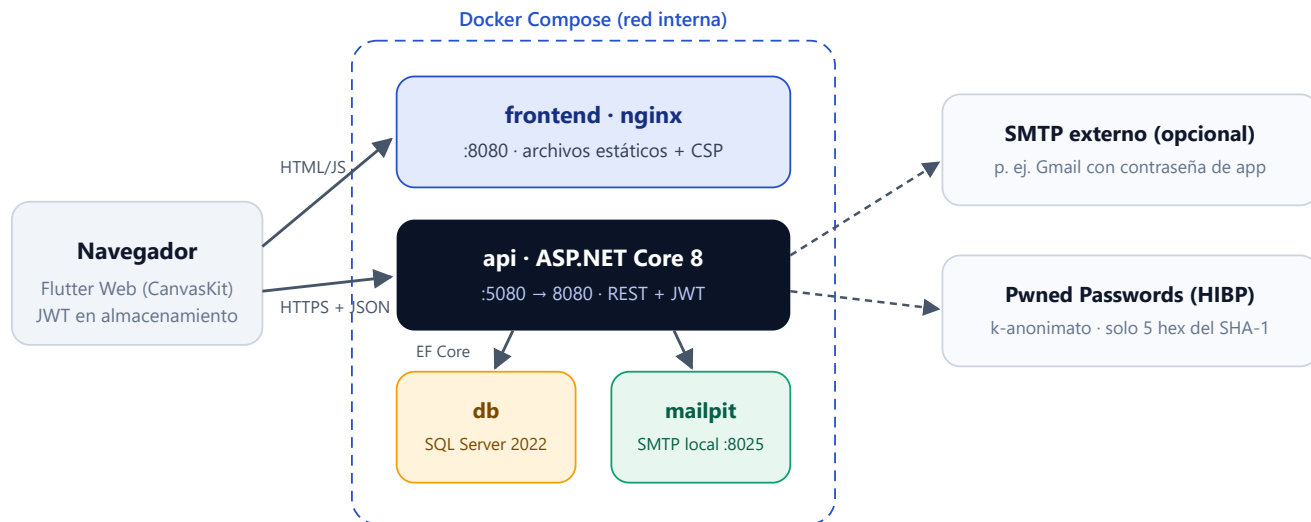
El sistema empezó como la prueba técnica *Sistema de Pedidos*: una API de pedidos con control de existencias y un catálogo web. La rama `feature/login-erp` lo extiende a un ERP para una comercializadora guatemalteca, con estos módulos:

- **Seguridad:** inicio de sesión por correo, verificación en dos pasos TOTP obligatoria, recuperación de contraseña, bitácora de accesos y administración de usuarios con invitación por correo.
- **Ventas:** facturación de contado con IVA incluido (FEL simulada), clientes con NIT validado, entrega por departamento y municipio, y pipeline de seis etapas.
- **Inventario:** catálogo con fotos y ficha técnica, existencias, costo promedio ponderado, kardex y ajustes.
- **Compras:** proveedores, órdenes de compra y recepción en bodega.
- **Contabilidad:** partidas automáticas y manuales, libro mayor, balance de comprobación, estado de resultados y balance general.
- **Reportes:** ventas con comparación contra el período anterior, mapa por departamento y pronóstico (forecast) por probabilidad de cierre.

Este manual describe cómo está construido el sistema y cómo instalarlo, operarlo y extenderlo. Para el uso funcional, consulta el **Manual de usuario**. El repositorio incluye además `README.md`, que explica cómo levantar y probar el sistema, y `SECURITY_REVIEW.md`, con la revisión de seguridad.

2 Arquitectura

Es una aplicación de tres capas, empaquetada en contenedores con Docker Compose:



La base de datos no publica puertos: solo la API la alcanza, por la red interna. Las líneas punteadas son conexiones salientes opcionales.

Flujo de una petición

- 1 El navegador descarga la app Flutter compilada desde nginx (:8080). La CSP de nginx solo permite llamadas al propio origen y a la URL de la API.
- 2 La app llama a la API (:5080) con `Authorization: Bearer <JWT>`.
- 3 En la API, la petición atraviesa en orden: manejo de excepciones → cabeceras de seguridad → CORS → autenticación JWT, que valida firma, emisor, audiencia, vigencia y **versión de sesión** contra la BD → límites de peticiones → autorización por rol → controlador.
- 4 El controlador delega en un **servicio** de dominio. El servicio usa EF Core sobre SQL Server, con transacciones y `UPDATE` condicionales para la concurrencia.
- 5 La respuesta es JSON. Los errores llegan siempre como `{ "error": "mensaje" }`, sin detalles internos.

Tareas en segundo plano de la API

- **Cola de notificaciones:** los correos (invitación, recuperación, avisos de seguridad) se encolan en memoria y los envía un `BackgroundService` con MailKit. Así una petición nunca espera al SMTP.
- **Generador de datos de demostración:** si `Demo_Generar=true`, al primer arranque crea productos, clientes y `Demo_Dias` días de ventas y compras con su contabilidad. Después sigue generando la actividad diaria de demostración.

3 Tecnologías y versiones

Componente	Tecnología	Notas
API	.NET 8 · ASP.NET Core (controladores)	SDK fijado en <code>backend/global.json</code> . Imagen <code>mcr.microsoft.com/dotnet/aspnet:8.0</code> , usuario sin privilegios.
ORM	Entity Framework Core 8.0.31	Proveedor SQL Server; SQLite en las pruebas unitarias.
Base de datos	SQL Server 2022 (Developer)	Esquema y semilla en <code>db/init/init.sql</code> (idempotente).
Autenticación	JWT HS256 · BCrypt.Net-Next 4.2.1 · Otp.NET 1.4.1	TOTP RFC 6238 compatible con Google Authenticator.
Correo	MailKit 4.18.1	HTML + texto plano (multipart/alternative).
Documentación API	Swashbuckle 6.6.2	Swagger solo en el entorno Development.
Frontend	Flutter 3.47.5 (Web) · Dart	Provider, http, intl, flutter_secure_storage, qr_flutter, file_picker, flutter_localizations (es-GT).
Servidor web	nginx-unprivileged 1.27 (Alpine)	Solo GET/HEAD, límites por IP, CSP estricta.
Correo local	Mailpit 1.31	Bandeja en <code>http://localhost:8025</code> .
Contenedores	Docker Compose	Contenedores de solo lectura, sin capacidades de Linux y con topes de memoria y CPU.

4 Estructura del repositorio

```
.
├── docker-compose.yml          # db, api, mailpit, frontend
├── .env.example                # plantilla de variables (el .env real no se versiona)
├── README.md · SECURITY_REVIEW.md · MENSAJE_PM.md
├── db/
│   ├── entrypoint.sh · healthcheck.sh · sqlcmd.sh
│   └── init/
│       ├── init.sql           # esquema + migraciones idempotentes + semillas + login de la app
│       └── imagenes/          # fotos semilla de productos (con CREDITOS.md)
├── backend/
│   ├── Dockerfile · Pedidos.sln · global.json
│   └── src/Pedidos.Api/
│       ├── Program.cs         # configuración, DI, middleware, JWT, CORS, rate limiting
│       ├── Controllers/       # Auth, Cuenta, Usuarios, Pedidos, Productos, ErpControllers
│       ├── Services/          # AuthService, PedidoService, Seguridad/, Usuarios/, Erp/
│       ├── Domain/            # entidades y constantes de negocio (Roles, EstadosVenta...)
│       ├── Data/AppDbContext.cs
│       ├── Dtos/ · Errors/ · Security/ · Notificaciones/
│       └── appsettings.json
│   └── tests/Pedidos.Tests/    # xUnit: unitarias (SQLite) + Http/ (API real + SQL Server)
├── frontend/
│   ├── Dockerfile · nginx.conf · pubspec.yaml
│   ├── assets/mapas/          # GeoJSON de departamentos (geoBoundaries, ODbL)
│   └── lib/
│       ├── main.dart · app.dart # composición de dependencias
│       ├── core/              # config, network (ApiClient), security, theme, util, widgets
│       ├── data/              # models/ y repositories/ (uno por área)
│       └── features/           # auth, cart, catalog, clientes, compras, contabilidad, cuenta,
│                               # inventario, order, panel, productos, reportes, shell, usuarios, ventas
└── test/                      # widget, layout responsive y escenarios de error
```

5 Instalación y despliegue

5.1 Requisitos

- Docker Desktop 4.x (o Docker Engine + Compose v2), con al menos 4 GB de RAM para los contenedores. SQL Server pide 2 GB.
- Para desarrollar sin Docker: .NET SDK 8, Flutter 3.47.5 y SQL Server (Express sirve).

5.2 Puesta en marcha con Docker

```
# 1. Variables de entorno (una sola vez)
cp .env.example .env          # y reemplazar contraseñas y claves
openssl rand -base64 48       # → JWT_KEY
openssl rand -base64 32       # → SEGURIDAD_CLAVE_MAESTRA

# 2. Construir y levantar
docker compose up -d --build

# 3. Verificar
docker compose ps              # db (healthy), api, frontend, mailpit
curl http://localhost:5080/health
```

Servicio	URL	Descripción
frontend	http://localhost:8080	Aplicación web.
api	http://localhost:5080	API REST. /health no requiere sesión.
mailpit	http://localhost:8025	Correos enviados por la API cuando no hay SMTP externo.
db	(sin puerto publicado)	SQL Server; solo accesible desde la red interna.

5.3 Variables de entorno (.env)

Variable	Uso
MSSQL_SA_PASSWORD	Contraseña de sa . Solo la usan los scripts de inicialización.
DB_NAME · APP_DB_USER · APP_DB_PASSWORD	Base de datos y login de mínimos privilegios con que se conecta la API. Lo crea init.sql .
JWT_KEY	Clave HS256 para firmar los tokens. Mínimo 32 caracteres; la API no arranca con una más corta.

<code>SEGURIDAD_CLAVE_MAESTRA</code>	32 bytes en Base64. De ella se derivan la clave AES que cifra los secretos TOTP y la clave HMAC de códigos y enlaces. Si se cambia, todos los usuarios deben volver a configurar Google Authenticator.
<code>CORREO_HOST</code> · <code>_PUERTO</code> · <code>_TLS</code> · <code>_USUARIO</code> · <code>_PASSWORD</code> · <code>_REMITENTE</code>	SMTP saliente. Sin estas variables se usa Mailpit. Para Gmail: <code>smtp.gmail.com</code> , puerto 465, <code>Ss10nConnect</code> y una contraseña de aplicación.
<code>API_PORT</code> · <code>API_PUBLIC_URL</code> · <code>FRONTEND_ORIGIN</code>	Puerto publicado de la API, URL con que la ve el navegador (se compila en el frontend y en su CSP) y origen permitido por CORS.
<code>DEMO_GENERAR</code> · <code>DEMO_DIAS</code>	Datos de demostración en el primer arranque y cuántos días de historia generar.

El archivo .env nunca se versiona

Contiene contraseñas y claves y está en `.gitignore`. En producción, guarda estos valores en el gestor de secretos de la plataforma.

5.4 Primer arranque

El contenedor `db` ejecuta `db/entrypoint.sh`: arranca SQL Server, espera a que responda y aplica `db/init/init.sql`. El script es **idempotente** (usa `IF OBJECT_ID... IS NULL`, `IF NOT EXISTS` y `EXEC` dinámico para las columnas nuevas), así que se puede reaplicar sobre una base existente para migrarla. El script:

- Crea las tablas, índices y restricciones, y los datos semilla: usuarios demo, productos con fotos, consumidor final, catálogo de cuentas, probabilidades del pipeline y la partida de apertura.
- Crea el login `APP_DB_USER` con permisos solo de lectura y escritura de datos (`db_datareader` y `db_datawriter`), sin DDL.

Después de actualizar `init.sql` en un ambiente existente, ejecuta `docker compose restart db`.

5.5 Desarrollo sin Docker

```
# API (user-secrets o variables de entorno con las mismas claves que el .env)
cd backend/src/Pedidos.Api
dotnet user-secrets set "ConnectionStrings:Default"
"Server=.\SQLEXPRESS;Database=PedidosDb;Trusted_Connection=True;TrustServerCertificate=True"
dotnet user-secrets set "Jwt:Key" "<32+ caracteres>"
dotnet user-secrets set "Seguridad:ClaveMaestra" "<32 bytes base64>"
dotnet run

# Frontend
cd frontend
flutter run -d chrome --dart-define=API_URL=http://localhost:5080 --no-web-resources-cdn
```

Rutas con tildes en Windows

Si la ruta del proyecto tiene caracteres como "á" o "ñ", `flutter analyze` puede fallar. Mapea la carpeta a una unidad con `subst P: "<ruta>"` y trabaja desde `P:\frontend`.

5.6 Operación diaria

Tarea	Comando
Ver logs de la API	<code>docker compose logs -f api</code>
Reconstruir tras cambiar código	<code>docker compose up -d --build api frontend</code>
Aplicar cambios de <code>init.sql</code>	<code>docker compose restart db</code>
Consola SQL	<code>docker compose exec db /opt/mssql-tools18/bin/sqlcmd -C -S localhost -U sa -P "\$MSSQL_SA_PASSWORD" -d PedidosDb</code>
Respaldo de la base	<code>BACKUP DATABASE PedidosDb TO DISK='/var/opt/mssql/backup/pedidos.bak' WITH INIT</code> y luego <code>docker compose cp db:/var/opt/mssql/backup/pedidos.bak .</code>
Restaurar	Copia el .bak al contenedor y ejecuta <code>RESTORE DATABASE PedidosDb FROM DISK='...' WITH REPLACE</code> (con la API detenida).
Empezar de cero (borra todos los datos)	<code>docker compose down -v</code> y <code>docker compose up -d --build</code>

6 Backend (API .NET 8)

6.1 Capas

Carpeta	Responsabilidad
Controllers/	HTTP: rutas, atributos <code>[Authorize]</code> , validación de forma y códigos de respuesta. No contienen reglas de negocio.
Services/	Reglas de negocio y transacciones: <code>PedidoService</code> , <code>AuthService</code> , <code>Seguridad/</code> (cuenta, 2FA, recuperación, bitácora), <code>Usuarios/</code> y <code>Erp/</code> (inventario, compras, contabilidad, terceros, pipeline, reportes, pronóstico, imágenes, geografía, panel, generador demo).
Domain/	Entidades EF y constantes: <code>Roles</code> , <code>EstadosVenta</code> (pipeline), <code>CuentasSistema</code> , <code>FormasPago</code> y tipos de movimiento.
Data/	<code>AppDbContext</code> : mapeo, índices, <i>check constraints</i> , conversiones (UTC, JSON) y semillas <code>HasData</code> .
Dtos/	Contratos de entrada y salida (records). Las entidades nunca se exponen directamente.
Security/	JWT, cifrado AES-GCM y HMAC, TOTP, política de contraseñas, límites de entrada y de peticiones, cabeceras de seguridad.
Notificaciones/	Cola de correos, envío SMTP y plantilla HTML de los correos.
Errors/	Excepciones de negocio y el manejador global que las traduce a respuestas HTTP.

6.2 Errores y códigos de respuesta

Código	Cuándo	Origen
400	Regla de negocio o datos inválidos	<code>BusinessRuleException</code> , o un cuerpo mal formado (mensaje genérico, sin detalles del parser).
401	Sin sesión, token inválido, vencido o revocado	Eventos de <code>JwtBearer</code> .
403	El rol no tiene permiso, o la cuenta está desactivada	Autorización por rol.
404	Recurso inexistente o ajeno	<code>NoEncontradoException</code> . Un vendedor que pide una venta de otro recibe 404, no 403, para no revelar que existe.
409	Conflicto de concurrencia	Violación de índice único (SQL 2601/2627) o víctima de bloqueo mutuo (1205).

413 · 415	Cuerpo demasiado grande · tipo de contenido no soportado	Kestrel (32 KB por defecto; 2 MB en la subida de fotos).
429	Límite de peticiones o cuenta bloqueada	Rate limiter (con <code>Retry-After</code>) y <code>LoginThrottle</code> .
500	Error inesperado	Se registra en el log del servidor; el cliente solo ve un mensaje genérico.

6.3 Configuración obligatoria

La API valida su configuración al arrancar y se detiene con un mensaje claro si falta algo: la cadena de conexión, `Jwt:Key` de 32 o más caracteres, `Seguridad:ClaveMaestra` de 32 bytes, o `Cors:AllowedOrigins` con orígenes explícitos (no se acepta `*`).

7 Seguridad

7.1 Autenticación

- **Contraseñas:** BCrypt. La política sigue NIST SP 800-63B: mínimo 12 caracteres, máximo 128, que no contenga el correo y que no aparezca en filtraciones (consulta a Pwned Passwords por k-anonimato; si el servicio no responde, la verificación se omite).
- **Sesión:** JWT HS256 de 60 minutos con `sub`, `rol` y `sv` (versión de sesión). Cada petición compara `sv` con `Usuarios.VersionSesion`. Cambiar la contraseña o el rol, desactivar al usuario o desactivar el 2FA incrementan la versión e invalidan al instante los tokens anteriores.
- **Dos pasos (TOTP):** obligatoria para todos los roles. La contraseña devuelve un *desafío*: un JWT de 5 minutos con otra audiencia, que no sirve como sesión. El secreto TOTP se guarda cifrado con AES-GCM. Se acepta una ventana de ± 30 s y un código ya usado se rechaza (anti-repetición). El desafío se invalida tras 5 códigos erróneos.
- **Respaldo y dispositivos:** 10 códigos de respaldo de un solo uso, guardados como HMAC-SHA256. "Confiar en este dispositivo" emite un token aleatorio de 30 días (en la BD solo se guarda su hash).
- **Recuperación e invitación:** tokens de 256 bits de un solo uso, guardados como HMAC. Vigencia de 30 minutos (recuperación) y 48 horas (invitación). `/recuperar` responde igual exista o no el correo.
- **Fuerza bruta:** 5 contraseñas erróneas bloquean la cuenta 5 minutos, además de los límites por IP.

7.2 Límites de peticiones

Política	Límite	Partición
Global (toda la API)	300 / minuto	IP
Login	10 / minuto	IP
Segundo factor, cuenta, restablecer, alta de usuarios	30 / minuto	IP
Recuperar contraseña	5 / 15 minutos	IP
Crear venta	20 / minuto (token bucket)	Usuario
nginx (frontend)	20 peticiones/s (ráfaga 60) · 30 conexiones	IP

Detrás de un proxy

Los límites usan la IP de la conexión. Si la API se publica detrás de un balanceador, configura `ForwardedHeaders` con la IP del proxy como confiable. De lo contrario, todos los usuarios compartirían un mismo cupo.

7.3 Otras defensas

- **Autorización por defecto:** la política *fallback* exige autenticación en todo endpoint, salvo los marcados `[AllowAnonymous]` (login, verificación, recuperación, restablecer, foto pública de producto y `/health`).
- **Cabeceras:** `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, `Permissions-Policy`, `Content-Security-Policy: default-src 'none'` y `Cache-Control: no-store`. Sin cabecera `Server`.
- **CORS:** solo el origen del frontend, con los métodos GET/POST/PUT/DELETE y las cabeceras `Authorization` y `Content-Type`.
- **Entradas:** el precio y el total los calcula siempre el servidor. Cuerpo máximo de 32 KB, hasta 50 líneas por documento y 1000 unidades por línea. Textos recortados y sin caracteres de control. Fotos validadas por su firma binaria (JPEG, PNG o WebP), no por la extensión.
- **Inyección SQL:** todas las consultas pasan por EF Core parametrizado, que en las búsquedas escapa los comodines `%` y `_` (verificado por las pruebas HTTP).
- **Contenedores:** sistema de archivos de solo lectura, `cap_drop: ALL`, `no-new-privileges`, usuarios sin privilegios y topes de memoria y CPU. La base de datos no se publica fuera de Docker.

8 Roles y permisos

Los roles son constantes en `Domain/Roles.cs`, con un espejo en `frontend/lib/core/security/permisos.dart`. Quien decide de verdad es la API: el frontend solo oculta lo que no se puede usar.

Grupo (constante)	Roles	Se usa en
<code>GestionClientes</code>	VENDEDOR, ADMIN	Clientes
<code>ConsultaVentas</code>	VENDEDOR, ADMIN, CONTADOR	Ventas, reportes y pronóstico (el vendedor solo ve lo suyo)
Crear venta	VENDEDOR	<code>POST /api/pedidos</code>
<code>Pipeline</code>	VENDEDOR, ADMIN, CONTADOR, BODEGA	Tablero y avance (además, cada etapa exige su rol)
<code>ConsultaInventario</code>	BODEGA, ADMIN, COMPRAS, CONTADOR	Existencias y kardex
<code>GestionInventario</code>	BODEGA, ADMIN	Productos, fotos y ajustes
<code>ConsultaCompras</code>	BODEGA, COMPRAS, ADMIN, CONTADOR	Proveedores y órdenes (lectura)
<code>GestionCompras</code>	COMPRAS, ADMIN	Crear proveedores, órdenes y anular
<code>RecepcionCompras</code>	BODEGA, COMPRAS, ADMIN	Recibir órdenes
<code>Contabilidad</code>	CONTADOR, ADMIN	Todo el módulo contable
<code>Panel</code>	ADMIN, BODEGA, COMPRAS, CONTADOR	Panel de inicio
<code>Admin</code>	ADMIN	Usuarios, bitácora y probabilidades del pronóstico

Etapas del pipeline y rol que las mueve

Hacia	Roles	Restricción
REVISADO	VENDEDOR, ADMIN	El vendedor solo sobre sus ventas
AUTORIZADO	CONTADOR, ADMIN	Un solo paso a la vez, sin retroceso.
DESPACHADO · EN_CAMINO · ENTREGADO	BODEGA, ADMIN	Cambio condicional: <code>UPDATE ... WHERE Estado = @anterior</code>

9 Reglas de negocio

9.1 Ventas

- **Precios con IVA (12 %) incluido.** La base imponible y el IVA se separan con `Montos.SepararIva`, de modo que $\text{base} + \text{IVA} = \text{total exacto al centavo}$.
- **Existencias:** cada línea descuenta con un `UPDATE ... SET Stock = Stock - @n WHERE Id = @id AND Stock >= @n` dentro de una transacción. Si alguna línea no alcanza, se revierte todo. La última unidad solo se puede vender una vez, aunque lleguen 20 pedidos simultáneos.
- **Costo de la venta:** el costo promedio vigente de cada producto se congela en la línea (`CostoUnitario`).
- **Factura FEL simulada:** serie "A", número = Id de la venta y autorización = GUID. No tiene validez fiscal.
- **Cientes:** NIT guatemalteco validado por módulo 11 (dígito 10 se escribe "K") y normalizado, con índice único. "CF" es el consumidor final del sistema.
- **Entrega:** departamento y municipio validados contra el catálogo oficial (22 departamentos y 340 municipios), sin distinguir tildes ni mayúsculas.

9.2 Inventario y compras

- **Costo promedio ponderado** en cada entrada: $(\text{stock} \times \text{costo} + \text{cantidad} \times \text{costoNuevo}) / (\text{stock} + \text{cantidad})$. Se calcula en el mismo `UPDATE` atómico.
- **Kardex** (`MovimientosInventario`): solo inserciones, con saldo y costo promedio después de cada movimiento.
- **Orden de compra:** PENDIENTE → RECIBIDA (exige la factura del proveedor; condicional, no se recibe dos veces) o ANULADA. Crear la orden no mueve el inventario.
- Un producto con historia no se elimina, solo se desactiva. Cada producto admite hasta 5 fotos de 2 MB como máximo.

9.3 Contabilidad automática

Cada operación genera su partida en la **misma transacción**. Si la partida falla, la operación tampoco se guarda.

Operación	Debe	Haber
Venta	1101 Caja (efectivo) o 1102 Bancos (tarjeta o transferencia) por el total; 5101 Costo de ventas	4101 Ventas (base); 2102 IVA por pagar; 1103 Inventario (costo)
Compra recibida	1103 Inventario (subtotal); 1104 IVA por cobrar	1102 Bancos (total, de contado)
Ajuste de entrada	1103 Inventario	4102 Otros ingresos

Ajuste de salida	6104 Faltantes y mermas	1103 Inventario
Apertura (semilla)	1102 Bancos; 1103 Inventario	3101 Capital social

Las partidas manuales deben cuadrar, tener al menos dos líneas (cada una solo en el debe o solo en el haber) y fecha no futura, y no admiten cuentas inactivas. Las fechas contables son `DateOnly` en hora de Guatemala (UTC-6). Los reportes admiten rangos de hasta un año.

9.4 Pronóstico (forecast)

La tabla `EtapasPipeline` guarda la probabilidad de cierre de cada etapa. Valores iniciales: 10, 25, 50, 75, 90 y 100 %. Solo ADMIN la cambia, y la API exige las seis etapas, con valores no decrecientes y ENTREGADO en 100 %.

```
esperado(etapa) = total(etapa) × probabilidad(etapa) / 100    (redondeado a centavos)
pronóstico      = cerrado (ENTREGADO) + Σ esperado(etapas abiertas)
avance de cierre = cerrado / pronóstico × 100
```

La tendencia se agrupa por día (hasta 31 días), por semana de lunes a domingo (hasta 120 días) o por mes.

10 Base de datos

Tabla	Contenido
Usuarios	Nombre, correo y código corporativo (únicos), teléfono, rol, hash BCrypt, estado del 2FA, secreto TOTP cifrado, VersionSesion y activo.
CodigosRespaldo · DispositivosConfiables · DispositivosConocidos · TokensRecuperacion	Seguridad de la cuenta (solo hashes HMAC); borrado en cascada con el usuario.
BitacoraAccesos	Eventos de seguridad con IP y navegador (textos limpiados de caracteres de control).
Productos · ProductoImagenes	Código único, precio, existencia (con <i>check</i> ≥ 0), mínimo, costo promedio, ficha (especificaciones en JSON); fotos en VARBINARY con orden.
Clientes · Proveedores	Terceros con NIT normalizado único.
Pedidos · PedidoDetalle · PedidoHistorial	Ventas: totales, IVA, costo, forma de pago, entrega y etapa (<i>check</i> de valores válidos); líneas con precio y costo congelados; historial de etapas.
EtapasPipeline	Probabilidad de cierre por etapa (0–100) y quién la cambió.
MovimientosInventario	Kardex (solo inserciones).
OrdenesCompra · OrdenCompraDetalle	Órdenes con estado, factura del proveedor y quién creó o recibió.
CuentasContables · Partidas · PartidaDetalle	Catálogo de cuentas y partidas con origen (VENTA, COMPRA, AJUSTE, MANUAL, APERTURA) y referencia.

Catálogo de cuentas semilla

1101 Caja · 1102 Bancos · 1103 Inventario de mercadería · 1104 IVA por cobrar · 2101 Proveedores · 2102 IVA por pagar · 3101 Capital social · 3102 Resultados acumulados · 4101 Ventas · 4102 Otros ingresos · 5101 Costo de ventas · 6101 Sueldos y salarios · 6102 Alquileres · 6103 Energía eléctrica, agua y teléfono · 6104 Faltantes y mermas · 6105 Gastos varios. Las cuentas que usan las partidas automáticas (CuentasSistema) no se pueden desactivar.

Convenciones

- Fechas y horas en UTC (DATETIME2), convertidas a hora de Guatemala para mostrarlas y para las fechas contables.
- Montos DECIMAL(18,2) ; las probabilidades, DECIMAL(5,2) .

- Cada cambio de esquema se agrega a `init.sql` de forma idempotente y a `AppDbContext`. Este último es el que usan las pruebas con `EnsureCreated`.

11 API REST: referencia de endpoints

Todo es JSON (UTF-8) bajo `/api`. Abreviaturas de roles: **V** vendedor, **A** admin, **B** bodega, **C** compras, **K** contador, **Todos** = cualquier usuario con sesión, **Público** = sin sesión.

Autenticación y cuenta

Método	Ruta	Roles	Descripción
POST	/api/auth/login	Público	Correo y contraseña → desafío de 2FA (o sesión, si el dispositivo es de confianza).
POST	/api/auth/login/verificar	Público	Desafío + código TOTP o de respaldo → JWT.
POST	/api/auth/recuperar	Público	Envía el enlace de recuperación (202 siempre).
POST	/api/auth/restablecer	Público	Token del enlace + nueva contraseña (también activa invitaciones).
GET	/api/cuenta/seguridad · /accesos	Todos	Estado del 2FA y actividad reciente.
POST	/api/cuenta/password	Todos	Cambia la contraseña; devuelve un token nuevo.
POST	/api/cuenta/2fa/totp · /totp/confirmar	Todos	Configura Google Authenticator en otro teléfono.
POST	/api/cuenta/2fa/codigos-respaldo · /desactivar	Todos	Regenera códigos (pide contraseña). Desactivar se rechaza: el 2FA es obligatorio.

Administración

Método	Ruta	Roles	Descripción
GET · POST	/api/admin/usuarios	A	Lista (?buscar=) y alta con invitación por correo.
GET · PUT · DELETE	/api/admin/usuarios/{id}	A	Detalle, edición (cambiar el rol cierra sesiones), eliminación (solo sin historia).
POST	/api/admin/usuarios/{id}/activar · /desactivar · /invitacion	A	Estado y reenvío de la invitación o del enlace.

GET	/api/admin/bitacora	A	Últimos eventos (?email=).
-----	---------------------	---	------------------------------

Ventas, clientes y pipeline

Método	Ruta	Roles	Descripción
GET	/api/productos · /{id}	Todos	Catálogo activo y ficha del producto.
GET	/api/productos/{id}/imagenes/{img}	Público	Foto (caché pública, Cross-Origin-Resource-Policy: cross-origin).
POST	/api/pedidos	V	{ lineas: [{productoId,cantidad}], clienteId?, formaPago?, direccionEntrega?, departamento?, municipio? } → 201.
GET	/api/pedidos · /{id}	V, A, K	Ventas del rango (?desde&hasta) y detalle con historial.
GET · POST · PUT	/api/clientes · /{id}	V, A	Búsqueda (?buscar&inactivos), alta y edición.
GET	/api/pipeline · /{id}	V, A, K, B	Tablero (indica a qué etapa puede llevar el usuario cada venta) y detalle.
POST	/api/pipeline/{id}/avanzar	V, A, K, B	{ nota? } → siguiente etapa, si al rol le toca.
GET	/api/geografia	Todos	Departamentos y municipios.

Inventario y compras

Método	Ruta	Roles	Descripción
GET	/api/inventario/productos	A, B, C, K	Existencias (?buscar&bajoMinimo).
POST · PUT · DELETE	/api/inventario/productos · /{id}	A, B	Alta, edición de ficha y eliminación (solo sin historia).
POST · DELETE	/api/inventario/productos/{id}/imagenes · /{img} · /{img}/principal	A, B	Fotos (multipart, campo archivo).
GET	/api/inventario/productos/{id}/kardex	A, B, C, K	Kardex.
POST	/api/inventario/ajustes	A, B	{ productoId, tipo: ENTRADA SALIDA, cantidad, costoUnitario?, motivo }.

GET · POST · PUT	/api/compras/proveedores · /{id}	Lectura A, B, C, K · escritura A, C	Proveedores.
GET · POST	/api/compras/ordenes · /{id}	Lectura A, B, C, K · alta A, C	Órdenes (?estado=).
POST	/api/compras/ordenes/{id}/recibir	A, B, C	{ facturaProveedor } .
POST	/api/compras/ordenes/{id}/anular	A, C	Solo pendientes.

Contabilidad, panel y reportes

Método	Ruta	Roles	Descripción
GET · POST · PUT	/api/contabilidad/cuentas · /{id}	A, K	Catálogo de cuentas.
GET · POST	/api/contabilidad/partidas · /{id}	A, K	Libro diario (? desde&hasta&origen) y partidas manuales.
GET	/api/contabilidad/reportes/mayor/{cuenta Id}	A, K	Libro mayor.
GET	/api/contabilidad/reportes/balance-comprobacion · /estado-resultados · /balance-general	A, K	Estados financieros (?desde&hasta o ?al).
GET	/api/panel	A, B, C, K	Indicadores del día y del mes.
GET	/api/reportes/ventas · /pronostico	V, A, K	Reporte de ventas y pronóstico (? desde&hasta).
PUT	/api/reportes/pronostico/probabilidades	A	{ etapas:[{estado, probabilidad}] } .

La prueba `AutorizacionHttpTests` comprueba, por reflexión, que esta lista coincide con los controladores.

12 Frontend (Flutter Web)

12.1 Capas

Carpeta	Contenido
core/	ApiClient , que agrega el token, traduce errores a ApiException y llama a onUnauthorized ante un 401. Además: tema, formato de moneda y fechas (es-GT) y widgets comunes (ModuloPage , TablaResponsiva con paginación, KpiCard , InlineBanner con Reintentar, selectores de rango y períodos).
data/	Modelos (fromJson) y repositorios por área. Son la única capa que conoce las rutas de la API.
features/	Pantallas y controladores por módulo. shell/modulos.dart define los grupos y módulos del menú y quién los ve.

12.2 Estado y navegación

- **Provider:** SessionController (sesión y flujo de 2FA), CatalogController , CartController (protegido contra doble envío) y NavegacionController (módulo resaltado).
- **Carga de datos:** las pantallas usan el mixin CargaDatos . Maneja los estados de carga, error y datos, ignora el 401 (la app vuelve al login) y atrapa respuestas con formato inesperado.
- **Navegación:** abrirModulo funciona como pestañas: reemplaza el módulo abierto, sin apilar pantallas.
- **Barra:** con 6 módulos o menos, van directos en la barra; con más, se agrupan por área. Debajo de 1240 px (1000 px si son directos), se usa un menú lateral.
- **Responsive:** puntos de quiebre 600 px (compacto) y 900 px (amplio). Las tablas se vuelven fichas y el kanban muestra una etapa a la vez.

12.3 Compilación

El Dockerfile instala Flutter 3.47.5 y compila con `flutter build web --release --no-web-resources-cdn --dart-define=API_URL=...` . La URL de la API queda dentro del bundle, por lo que **nada secreto debe ir en AppConfig** . nginx sirve los archivos con la CSP, en la que se sustituye la URL de la API.

13 Pruebas automatizadas

```
cd backend && dotnet test      # 181 pruebas (SQLite en memoria)
# con SQL Server: + pruebas HTTP contra la API real → 365
PEDIDOS_TEST_SQLSERVER='Server=.\SQLEXPRESS;Trusted_Connection=True;TrustServerCertificate=True' dotnet
test
cd frontend && flutter test     # 486 pruebas
```

Suite	Qué cubre
Unitarias del backend	Reglas de cada servicio sobre SQLite (transacciones reales, <i>check constraints</i>): ventas, existencias, costo promedio, compras, contabilidad cuadrada, NIT, IVA, pipeline, pronóstico, 2FA, recuperación, usuarios, correos y fotos.
HTTP contra la API real	Arrancan <code>Pedidos.Api.dll</code> en modo producción sobre una base temporal, con SMTP y Pwned Passwords simulados. Cubren la matriz de permisos completa, tokens alterados o vencidos, entradas maliciosas, CORS y cabeceras, flujos completos entre roles, concurrencia real y límites de peticiones.
Concurrencia (SQL Server)	Últimas unidades, la misma venta movida o la misma orden recibida por varias personas, y duplicados simultáneos.
Frontend	Repositorios y controladores, layout sin desbordes de 320 a 1920 px, arrastre del pipeline, escenarios de error en cada pantalla (500, 502, sin red, formato inesperado, 401, 403, 429, lentitud), login con el servidor caído y formularios.

14 Mantenimiento, extensión y solución de problemas

14.1 Agregar un endpoint

- 1 DTOs en `Dtos/` y la regla en un servicio (lanza `BusinessRuleException` o `NoEncontradoException`).
- 2 Método del controlador con su `[Authorize(Roles = ...)]` usando las constantes de `Roles`.
- 3 Agrégalo a la matriz de `AutorizacionHttpTests.Endpoints` con los roles esperados (la prueba de reflexión falla si falta).
- 4 Pruebas unitarias del servicio y, si aplica, un flujo HTTP.

14.2 Agregar un módulo al frontend

- 1 Modelo y método en el repositorio de `data/`.
- 2 Pantalla con `ModuloPage` + `CargaDatos` (los errores y **Reintentar** salen solos).
- 3 Entrada en `modulos.dart` (grupo, ícono y permiso de `permisos.dart`). La prueba de escenarios de error la recorre automáticamente.

14.3 Cambios de base de datos

Agrega la tabla o columna en `AppDbContext` y, de forma idempotente, en `init.sql`. Para una columna nueva que el mismo lote actualiza, usa `EXEC(N'...')`, porque SQL Server compila el lote completo antes de ejecutarlo.

14.4 Rotación de secretos

- **JWT_KEY**: al cambiarla se cierran todas las sesiones.
- **SEGURIDAD_CLAVE_MAESTRA**: deja inservibles los secretos TOTP, los códigos de respaldo y los enlaces pendientes. Antes de cambiarla, resetea el 2FA de todos (`DosFactor='NINGUNO'`, secretos en NULL) para que lo configuren de nuevo.
- **Contraseña SMTP**: actualiza el `.env` y ejecuta `docker compose up -d api`.

14.5 Problemas comunes

Síntoma	Causa y solución
<code>failed to connect to the docker API ... dockerDesktopLinuxEngine</code>	Docker Desktop no está corriendo. Ábrelo y espera a que el motor responda (<code>docker version</code>). Si se cuelga: cierra Docker, ejecuta <code>ws1 --shutdown</code> y vuelve a abrirlo.
La API se detiene al arrancar	Falta una variable obligatoria o es inválida. <code>docker compose logs api</code> muestra cuál.

Errores de columnas inexistentes tras actualizar	El volumen de la base tiene un esquema anterior: ejecuta <code>docker compose restart db</code> para aplicar <code>init.sql</code> .
No llegan correos	Sin <code>CORREO_*</code> llegan a Mailpit (:8025). Con Gmail, revisa la contraseña de aplicación y el puerto 465 con <code>SslOnConnect</code> .
La app web carga en blanco	<code>API_URL</code> no coincide con la URL real (CSP o CORS). Reconstruye el frontend con la URL correcta.
Todos los usuarios reciben 429	La API está detrás de un proxy sin <code>ForwardedHeaders</code> (ver 7.2).
Un usuario perdió el teléfono y sus códigos	Un administrador de base de datos resetea su 2FA (<code>DosFactor='NINGUNO'</code> , secretos en NULL, borra códigos y dispositivos, <code>VersionSesion+1</code>). En el siguiente ingreso configura la app de nuevo.